Multi-Agent AI-Based QoE Analysis for 5G Networks using LangChain and LangGraph

Laboratory Manual

Department of Computer Science and Engineering

February 4, 2026

Contents

1 Introduction **4**
1.1 Overview . . . 4
1.2 What You Will Learn . . . 4

2 Prerequisites and Setup **4**
2.1 Required Knowledge . . . 4
2.2 Software Requirements . . . 5
2.3 Installation Instructions . . . 5
2.3.1 Step 1: Install Python Packages . . . 5
2.3.2 Step 2: Install Ollama . . . 5
2.3.3 Step 3: Set Up LangSmith (Optional) . . . 5

3 Theoretical Background **6**
3.1 5G Network Quality Metrics . . . 6
3.1.1 Key Performance Indicators (KPIs) . . . 6
3.1.2 Quality of Experience (QoE) . . . 6
3.2 Multi-Agent AI Systems . . . 7
3.2.1 What is an Agent? . . . 7
3.2.2 Why Multi-Agent Architecture? . . . 7
3.3 LangChain and LangGraph . . . 7
3.3.1 LangChain . . . 7
3.3.2 LangGraph . . . 7

4 System Architecture **8**
4.1 Agent Workflow Overview . . . 8
4.2 LangGraph State Diagram . . . 8
4.3 Data Flow . . . 9

5 Implementation Guide **9**
5.1 Step 1: Environment Configuration . . . 9
5.1.1 Setting Up LangSmith Tracing . . . 9
5.2 Step 2: Import Required Libraries . . . 9
5.3 Step 3: Define State Structure . . . 10
5.4 Step 4: Initialize the Language Model . . . 10
5.5 Step 5: Implement KPI Sanity Check Agent . . . 11
5.6 Step 6: Implement QoE Classification Agent . . . 12
5.7 Step 7: Implement Network Advice Agent . . . 14
5.8 Step 8: Implement Human Review and Memory Agents . . . 15
5.9 Step 9: Build the LangGraph Workflow . . . 16
5.10 Step 10: Visualize the Workflow . . . 16
5.11 Step 11: Create Output Formatter . . . 17
5.12 Step 12: Run the System . . . 17

6 LangSmith Monitoring and Debugging **18**
6.1 Accessing LangSmith Dashboard . . . 18
6.2 LangSmith Output Screenshot . . . 18
6.3 What to Look For in LangSmith . . . 19

7 Experimental Tasks **19**
7.1 Task 1: Test Different Scenarios . . . 19
7.1.1 Scenario A: Excellent Connection . . . 19
7.1.2 Scenario B: Moderate Performance . . . 19
7.1.3 Scenario C: Invalid Input . . . 20
7.2 Task 2: Modify Few-Shot Examples . . . 20
7.3 Task 3: Enhance the Advice Agent . . . 20
7.4 Task 4: Implement Real Human Review . . . 20
7.5 Task 5: Add a New Agent . . . 21

8 Extensions and Future Work **21**
8.1 Potential Enhancements . . . 21
8.2 Research Directions . . . 22

9 Troubleshooting Guide **23**
9.1 Common Issues and Solutions . . . 23

10 Conclusion **24**
10.1 Key Takeaways . . . 24
10.2 Next Steps . . . 24

11 References and Resources **25**
11.1 Official Documentation . . . 25
11.2 Tutorials and Guides . . . 25
11.3 Community Resources . . . 25

A Appendix A: Sample Outputs **25**
A.1 Example Run Output . . . 25

B Appendix B: Glossary **26**

1 Introduction

1.1 Overview

This laboratory manual provides a comprehensive guide to building a multi-agent AI system for analyzing Quality of Experience (QoE) in 5G networks. The system uses LangChain, LangGraph, and large language models (LLMs) to process network Key Performance Indicators (KPIs) and provide intelligent recommendations.

1.2 What You Will Learn

By completing this lab, you will:

- Understand the fundamentals of 5G network performance metrics
- Learn how to design multi-agent AI systems using LangGraph
- Implement state management in agentic workflows
- Use LangChain for prompt engineering and LLM orchestration
- Apply few-shot learning techniques for classification tasks
- Build conditional workflows with human-in-the-loop validation
- Visualize and debug agent workflows using LangSmith

Lab Objectives

Main Goal: Create an automated system that takes 5G network KPIs as input and produces:

1. KPI validation results
2. QoE classification (Smooth/Acceptable/Poor)
3. Actionable network improvement advice

2 Prerequisites and Setup

2.1 Required Knowledge

Before starting this lab, you should have:

- Basic Python programming skills
- Understanding of functions and dictionaries
- Familiarity with Jupyter notebooks
- Basic understanding of AI/ML concepts (helpful but not required)

2.2 Software Requirements

1. **Python 3.8+** - Programming language
2. **Jupyter Notebook** - Interactive development environment
3. **Ollama** - Local LLM runtime (with llama3.1:8b model)
4. **LangChain** - Framework for LLM applications
5. **LangGraph** - Library for building stateful agent workflows
6. **LangSmith** - Monitoring and debugging tool (optional but recommended)

2.3 Installation Instructions

2.3.1 Step 1: Install Python Packages

Open your terminal/command prompt and run:

```
pip install langchain langchain-ollama langgraph langchain-core
```

2.3.2 Step 2: Install Ollama

1. Visit: https://ollama.com
2. Download and install Ollama for your operating system
3. Open terminal and pull the required model:

```
ollama pull llama3.1:8b
```

2.3.3 Step 3: Set Up LangSmith (Optional)

1. Create account at: https://smith.langchain.com
2. Navigate to Settings → API Keys
3. Create a new API key
4. Copy the key for use in the lab

Important Note

Keep your LangSmith API key secure and never share it publicly. For this lab, we'll set it as an environment variable.

3 Theoretical Background

3.1 5G Network Quality Metrics

3.1.1 Key Performance Indicators (KPIs)

5G networks are evaluated using several critical metrics:

1. **RSSI (Received Signal Strength Indicator)**
   - Measures signal power in dBm
   - Valid range: -40 dBm to -140 dBm
   - Closer to 0 = stronger signal
   - Example: -70 dBm is stronger than -110 dBm
2. **Latency (Network Delay)**
   - Time for data to travel from source to destination
   - Measured in milliseconds (ms)
   - Lower is better (critical for gaming, video calls)
   - 5G target: $<$ 10 ms for ultra-reliable applications
3. **Packet Loss**
   - Percentage of data packets that fail to reach destination
   - Valid range: 0% to 100%
   - Lower is better
   - Even 1-2% can impact real-time applications
4. **Network Type**
   - 4G (LTE) - Fourth generation
   - 5G NSA (Non-Standalone) - 5G using 4G core network
   - 5G SA (Standalone) - Pure 5G with 5G core network

3.1.2 Quality of Experience (QoE)

QoE represents the user's subjective perception of network performance:

- **Smooth:** Excellent performance, no noticeable issues
- **Acceptable:** Usable but with minor degradation
- **Poor:** Frustrating experience, significant issues

3.2 Multi-Agent AI Systems

3.2.1 What is an Agent?

An **agent** is an autonomous software component that:

- Receives input
- Performs specific tasks
- Makes decisions
- Produces output
- May use AI/LLMs for reasoning

3.2.2 Why Multi-Agent Architecture?

Instead of one monolithic AI system, we use multiple specialized agents:

Advantage	**Explanation**
Separation of Concerns	Each agent has one clear responsibility
Easier Debugging	Isolate which agent is causing issues
Modularity	Replace or improve individual agents
Specialization	Each agent optimized for its specific task
Scalability	Add new agents without rewriting everything

Table 1: Benefits of Multi-Agent Systems

3.3 LangChain and LangGraph

3.3.1 LangChain

LangChain is a framework that simplifies:

- Connecting to LLMs (OpenAI, Ollama, etc.)
- Managing prompts and prompt templates
- Parsing LLM outputs
- Building chains of LLM operations

3.3.2 LangGraph

LangGraph extends LangChain with:

- **State management:** Maintain data across agent steps
- **Graph-based workflows:** Define agent sequences visually
- **Conditional routing:** Different paths based on outputs
- **Human-in-the-loop:** Pause for human approval

Theoretical Background

Key Concept - State: In LangGraph, *state* is a dictionary that gets passed between agents. Each agent can read from and write to the state, enabling information flow through the workflow.

4 System Architecture

4.1 Agent Workflow Overview

Our system consists of 5 specialized agents working in sequence:

1. **KPI Sanity Check Agent** - Validates input data ranges
2. **QoE Classification Agent** - Categorizes user experience
3. **Human Review Agent** - Allows manual verification
4. **Network Advice Agent** - Generates recommendations
5. **Memory Agent** - Stores interaction history

4.2 LangGraph State Diagram

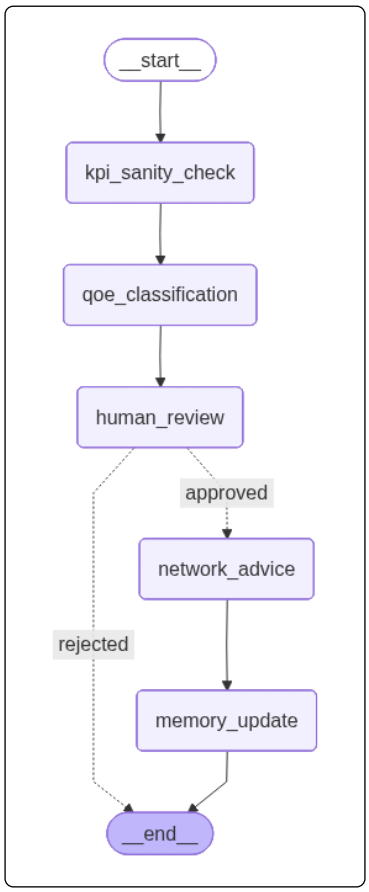

Figure 1: LangGraph Multi-Agent Workflow State Diagram

4.3 Data Flow

The state dictionary flows through agents with this structure:

```
class QoEState(TypedDict):
    user_input: str                # Original KPI data
    kpi_validation: str            # "VALID" or "INVALID" + reason
    qoe_classification: str        # "Smooth", "Acceptable", or "
        Poor"
    network_advice: str            # User-friendly recommendations
    memory: List[str]              # Historical interactions
    human_approved: bool           # Approval flag
```

Listing 1: QoEState Type Definition

5 Implementation Guide

5.1 Step 1: Environment Configuration

5.1.1 Setting Up LangSmith Tracing

LangSmith allows you to monitor every LLM call, agent transition, and debug issues.

```
import os

os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_PROJECT"] = "5G-QoE-Lang-Lab"
os.environ["LANGCHAIN_API_KEY"] = "your-api-key-here"
```

Listing 2: Environment Setup Code

Explanation:

- `LANGCHAIN_TRACING_V2`: Enables detailed logging
- `LANGCHAIN_PROJECT`: Organizes traces in LangSmith dashboard
- `LANGCHAIN_API_KEY`: Your authentication key

Exercise

Task 1: Replace `"your-api-key-here"` with your actual LangSmith API key from https://smith.langchain.com

5.2 Step 2: Import Required Libraries

```
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, SystemMessage
from langgraph.graph import StateGraph, START, END
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from typing import TypedDict, List
```

Listing 3: Import Statements

Component Explanation:

- `ChatOllama`: Interface to local Ollama LLM
- `HumanMessage/SystemMessage`: Structures for LLM conversations
- `StateGraph`: Builds agent workflow graphs
- `ChatPromptTemplate`: Creates reusable prompt templates
- `StrOutputParser`: Extracts string responses from LLM
- `TypedDict`: Defines typed state structure

5.3 Step 3: Define State Structure

```python
class QoEState(TypedDict):
    user_input: str
    kpi_validation: str
    qoe_classification: str
    network_advice: str
    memory: List[str]
    human_approved: bool
```

Listing 4: State Type Definition

Why TypedDict?

- Provides type hints for better code completion
- Documents expected state fields
- Helps catch errors during development
- Not strictly enforced at runtime but aids development

5.4 Step 4: Initialize the Language Model

```python
llm = ChatOllama(
    model="llama3.1:8b",
    temperature=0
)
```

Listing 5: LLM Initialization

Parameters Explained:

- `model="llama3.1:8b"`: Which LLM to use (8 billion parameters)
- `temperature=0`: Controls randomness
  - 0 = deterministic, always same output
  - 1 = creative, varied outputs
  - We use 0 for consistent, predictable classifications

5.5 Step 5: Implement KPI Sanity Check Agent

This agent validates whether KPI values are physically possible, without judging quality.

```python
def kpi_sanity_agent(state: QoEState) -> QoEState:
    system_prompt = """
You are a strict telecom KPI validation agent.

Your ONLY task:
- Check whether KPI VALUES are within allowed numeric ranges.
- Do NOT judge performance quality.
- Do NOT decide whether values are good or bad.
- High latency or weak signal can still be VALID.

Validation rules (strict):
- RSSI is VALID if between -40 dBm and -140 dBm (inclusive)
- Latency is VALID if it is a positive number (> 0 ms)
- Packet loss is VALID if between 0% and 100% (inclusive)
- Network type must be exactly one of: 4G, 5G NSA, 5G SA

Output format (STRICT):
VALID or INVALID
One-line reason ONLY if INVALID.
"""

    messages = [
        SystemMessage(content=system_prompt),
        HumanMessage(content=state["user_input"])
    ]

    response = llm.invoke(messages)

    return {
        "user_input": state["user_input"],
        "kpi_validation": response.content
    }
```

Listing 6: KPI Sanity Check Agent Implementation

Design Decisions:

1. **Strict separation:** This agent ONLY validates ranges, not quality
2. **Clear rules:** Explicit range specifications prevent hallucination
3. **Message pattern:** System message sets behavior, human message provides data
4. **State return:** Preserves input and adds validation result

Exercise

Task 2: Why do we insist the agent not judge quality? What could go wrong if validation and classification were combined in one agent?

5.6 Step 6: Implement QoE Classification Agent

This agent uses few-shot learning to classify user experience based on latency and packet loss.

```
qoe_prompt_lcel = ChatPromptTemplate.from_messages(
    [
        ("system", """
You are a telecom Quality of Experience (QoE) classification agent.

Your task:
- Classify user experience into exactly ONE of the following labels:
  Smooth, Acceptable, Poor

Base your decision ONLY on:
- Latency (ms)
- Packet loss (%)

Do NOT validate data.
Do NOT give network advice.
Do NOT invent new labels.

---

### Few-Shot Examples

Example 1:
Latency: 8 ms
Packet Loss: 0.2 %
QoE: Smooth

Example 2:
Latency: 15 ms
Packet Loss: 0.5 %
QoE: Smooth

Example 3:
Latency: 25 ms
Packet Loss: 0.9 %
QoE: Smooth

[... more examples ...]

Example 15:
Latency: 20 ms
Packet Loss: 0.3 %
QoE: Smooth
"""),
        ("human", "{user_input}")
    ]
```

```
)
```

Listing 7: QoE Prompt Template with Few-Shot Examples

Few-Shot Learning Explanation:
Few-shot learning teaches the LLM by example rather than explicit rules:

- **0-shot:** No examples, agent must infer from instructions
- **Few-shot:** 5-20 examples showing input→output patterns
- **Many-shot:** 50+ examples (not needed for simple tasks)

Our examples establish implicit decision boundaries:

- Latency $<$ 30ms + Loss $<$ 1% → Smooth
- Latency 30-100ms + Loss 1-3% → Acceptable
- Latency $>$ 100ms or Loss $>$ 3% → Poor

```
qoe_chain_lcel = qoe_prompt_lcel | llm | StrOutputParser()
```

Listing 8: Creating LCEL Chain

LCEL (LangChain Expression Language):

- The `|` operator chains components
- Prompt → LLM → Parser creates a reusable pipeline
- Can be invoked with `.invoke({variables})`

```
def qoe_classification_agent(state: QoEState) -> QoEState:
    qoe_result = qoe_chain_lcel.invoke(
        {"user_input": state["user_input"]}
    )

    return {
        "user_input": state["user_input"],
        "kpi_validation": state["kpi_validation"],
        "qoe_classification": qoe_result
    }
```

Listing 9: QoE Classification Agent

Exercise

Task 3: Design 3 additional few-shot examples for edge cases (e.g., very high latency, medium packet loss).

5.7 Step 7: Implement Network Advice Agent

```python
def network_advice_agent(state: QoEState) -> QoEState:
    system_prompt = """
You are a telecom network advisory agent.

Your task:
- Provide short, clear advice to the user based on QoE classification.
- Use simple language, as if speaking to a non-expert user.

Rules:
- If QoE is Smooth -> reassure the user.
- If QoE is Acceptable -> suggest minor improvements.
- If QoE is Poor -> suggest clear corrective actions.

Do NOT:
- Reclassify QoE
- Validate KPIs
- Mention internal reasoning or few-shot examples

Advice examples:
- Suggest changing location, switching network mode,
  checking congestion, or contacting the operator
  when appropriate.
"""

    messages = [
        SystemMessage(content=system_prompt),
        HumanMessage(
            content=f"""
User KPIs:
{state['user_input']}

QoE Classification:
{state['qoe_classification']}
"""
        )
    ]

    response = llm.invoke(messages)

    return {
        "user_input": state["user_input"],
        "kpi_validation": state["kpi_validation"],
        "qoe_classification": state["qoe_classification"],
        "network_advice": response.content
    }
```

Listing 10: Network Advice Agent

Key Features:

- Receives both raw KPIs and classification
- Generates actionable, user-friendly advice
- Tailors recommendations to QoE level
- Avoids technical jargon

5.8 Step 8: Implement Human Review and Memory Agents

```
def human_review(state: QoEState):
    state["human_approved"] = True
    return state
```

Listing 11: Human Review Agent

Note: In production, this would pause for actual human input. For this lab, it auto-approves.

```
def route_after_review(state: QoEState):
    if state["human_approved"]:
        return "approved"
    else:
        return "rejected"
```

Listing 12: Routing Function

This enables conditional workflow branching based on approval status.

```
def memory_agent(state: QoEState) -> QoEState:
    summary = f"""
QoE: {state['qoe_classification']}
Advice: {state['network_advice']}
"""

    updated_memory = state["memory"] + [summary.strip()]

    return {
        **state,
        "memory": updated_memory
    }
```

Listing 13: Memory Agent

Purpose:

- Maintains conversation history
- Enables multi-turn interactions (future enhancement)
- Could be used for trend analysis

5.9 Step 9: Build the LangGraph Workflow

```
workflow = StateGraph(QoEState)

# Add all agent nodes
workflow.add_node("kpi_sanity_check", kpi_sanity_agent)
workflow.add_node("qoe_classification", qoe_classification_agent)
workflow.add_node("human_review", human_review)
workflow.add_node("network_advice", network_advice_agent)
workflow.add_node("memory_update", memory_agent)

# Define sequential flow
workflow.add_edge(START, "kpi_sanity_check")
workflow.add_edge("kpi_sanity_check", "qoe_classification")
workflow.add_edge("qoe_classification", "human_review")

# Conditional branching after human review
workflow.add_conditional_edges(
    "human_review",
    route_after_review,
    {
        "approved": "network_advice",
        "rejected": END
    }
)

# Continue to memory and end
workflow.add_edge("network_advice", "memory_update")
workflow.add_edge("memory_update", END)

# Compile the graph
app = workflow.compile()
```

Listing 14: Workflow Construction

Graph Components:

- `add_node()`: Registers an agent function
- `add_edge()`: Creates unconditional transition
- `add_conditional_edges()`: Branches based on function output
- `START/END`: Special nodes marking workflow boundaries
- `compile()`: Finalizes the graph for execution

5.10 Step 10: Visualize the Workflow

```
from IPython.display import display, Image

display(Image(app.get_graph().draw_mermaid_png()))
```

Listing 15: Generate State Diagram

This generates the diagram you should include in Figure 1.

5.11 Step 11: Create Output Formatter

```
def format_final_output(result: dict):
    print("=" * 60)
    print("5G NETWORK QUALITY ANALYSIS REPORT")
    print("=" * 60)

    print("\nKPI SANITY CHECK")
    print(result["kpi_validation"])

    print("\nQoE CLASSIFICATION")
    print(result["qoe_classification"])

    print("\nNETWORK ADVICE")
    print(result["network_advice"])

    print("\nFINAL VERDICT")
    if "Poor" in result["qoe_classification"]:
        print("User experience is currently NOT satisfactory.")
    elif "Acceptable" in result["qoe_classification"]:
        print("User experience is usable but can be improved.")
    else:
        print("User experience is smooth and satisfactory.")

    print("=" * 60)
```

Listing 16: Result Formatting Function

5.12 Step 12: Run the System

```
student_input = """
RSSI: -110 dBm
Latency: 180 ms
Packet Loss: 4.5 %
Network Type: 5G NSA
"""

result = app.invoke({
    "user_input": student_input,
    "memory": [],
    "human_approved": False
})

format_final_output(result)
```

Listing 17: Example Execution

Expected Flow:

1. KPI Sanity: Validates ranges → "VALID"
2. QoE Classification: Analyzes metrics → "Poor"
3. Human Review: Auto-approves → "approved"
4. Network Advice: Generates recommendations
5. Memory: Stores interaction
6. Returns final state dictionary

6 LangSmith Monitoring and Debugging

6.1 Accessing LangSmith Dashboard

After running your workflow:

1. Visit https://smith.langchain.com
2. Navigate to your project: "5G-QoE-Lang-Lab"
3. View the latest trace

6.2 LangSmith Output Screenshot

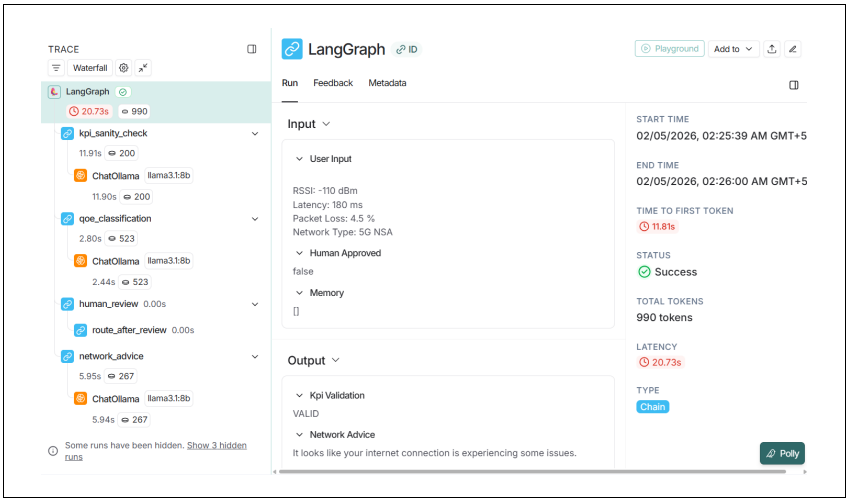

Figure 2: LangSmith Execution Trace

6.3 What to Look For in LangSmith

Feature	Usage
Trace Timeline	See total execution time and bottlenecks
Agent Transitions	Verify workflow follows expected path
LLM Input/Output	Inspect exact prompts and responses
Token Usage	Monitor costs (important for paid APIs)
Error Messages	Debug failures with full stack traces
Latency Metrics	Identify slow agents for optimization

Table 2: LangSmith Dashboard Features

Exercise

Task 4: Run your workflow and capture a screenshot of the LangSmith trace. Identify:

- Which agent took the longest time?
- How many total tokens were used?
- What was the exact prompt sent to the QoE classification agent?

7 Experimental Tasks

7.1 Task 1: Test Different Scenarios

Run the system with these inputs and record outputs:

7.1.1 Scenario A: Excellent Connection

```
RSSI: -65 dBm
Latency: 12 ms
Packet Loss: 0.1 %
Network Type: 5G SA
```

7.1.2 Scenario B: Moderate Performance

```
RSSI: -95 dBm
Latency: 55 ms
Packet Loss: 1.8 %
Network Type: 5G NSA
```

7.1.3 Scenario C: Invalid Input

```
RSSI: -200 dBm
Latency: -5 ms
Packet Loss: 150 %
Network Type: 6G
```

7.2 Task 2: Modify Few-Shot Examples

Exercise

Challenge: The current system may misclassify edge cases. Add 5 new few-shot examples that help the model better distinguish between "Acceptable" and "Poor" in the 80-120ms latency range.

7.3 Task 3: Enhance the Advice Agent

Exercise

Challenge: Modify the network advice agent to also consider:

- RSSI strength (weak signal suggestions)
- Network type (suggest upgrading to 5G SA)
- Time-of-day considerations (network congestion)

7.4 Task 4: Implement Real Human Review

Currently, the human review agent auto-approves. Modify it to:

```python
def human_review(state: QoEState):
    print("\n" + "="*50)
    print("HUMAN REVIEW REQUIRED")
    print("="*50)
    print(f"\nKPI Validation: {state['kpi_validation']}")
    print(f"QoE Classification: {state['qoe_classification']}")

    user_input = input("\nApprove this classification? (yes/no): "
        )
    state["human_approved"] = user_input.lower() == "yes"

    return state
```

Listing 18: Interactive Human Review

Test this with multiple scenarios and document cases where you disagreed with the AI.

7.5 Task 5: Add a New Agent

Exercise

Advanced Challenge: Design and implement a "Trend Analysis Agent" that:

1. Analyzes the memory list
2. Detects patterns (e.g., consistently degrading QoE)
3. Generates long-term recommendations

8 Extensions and Future Work

8.1 Potential Enhancements

1. **Multi-Modal Analysis**
   - Include throughput, jitter, MOS scores
   - Process signal heatmaps using vision models
   - Integrate user feedback sentiment analysis
2. **Contextual Awareness**
   - Consider location (urban vs rural)
   - Time-based patterns (peak hours)
   - Device type and application requirements
   - Weather conditions affecting signal
3. **Predictive Capabilities**
   - Forecast degradation trends
   - Proactive handover recommendations
   - Capacity planning for network operators
4. **Advanced Agent Features**
   - Self-reflection: Agent reviews its own outputs
   - Tool use: Agents query live network databases
   - Multi-agent collaboration: Parallel analysis with voting
   - Retrieval-Augmented Generation (RAG): Knowledge base integration
5. **User Interface**
   - Web dashboard with graph visualization
   - Mobile app for field technicians
   - Voice interface for hands-free operation
   - Integration with network management systems

8.2 Research Directions

- Compare LLM-based vs traditional ML classifiers
- Benchmark different LLMs (GPT-4, Claude, Llama)
- Optimize prompt engineering through automated testing
- Develop domain-specific fine-tuned models

9 Troubleshooting Guide

9.1 Common Issues and Solutions

Problem	**Solution**
"Ollama connection refused"	• Ensure Ollama is running: `ollama serve` • Check if model is pulled: `ollama list` • Verify port 11434 is accessible
"LangSmith API key invalid"	• Regenerate key in LangSmith dashboard • Check for extra spaces in API key string • Set environment variable directly in terminal
"Import errors for LangChain"	• Update packages: `pip install -upgrade langchain langgraph` • Check Python version (need 3.8+) • Install missing dependencies individually
"Agent returns unexpected format"	• Review prompt clarity • Add more few-shot examples • Increase temperature slightly (0.1) • Check LangSmith trace for actual LLM output
"Workflow hangs indefinitely"	• Check for infinite loops in routing • Verify all edges lead to END • Inspect LangSmith for last completed agent • Add timeout to LLM calls

Table 3: Troubleshooting Reference

10 Conclusion

In this lab, you have:

- Built a sophisticated multi-agent AI system
- Learned to orchestrate LLMs using LangChain and LangGraph
- Applied prompt engineering and few-shot learning techniques
- Implemented state management in agentic workflows
- Gained hands-on experience with LangSmith monitoring
- Explored practical applications of AI in telecommunications

10.1 Key Takeaways

1. **Modularity Matters:** Specialized agents are easier to debug and improve than monolithic systems
2. **Prompt Engineering is Critical:** Clear, specific prompts with examples dramatically improve output quality
3. **State Management Enables Complexity:** LangGraph's state passing allows sophisticated multi-step workflows
4. **Observability is Essential:** LangSmith-style tracing is crucial for debugging and optimization
5. **AI Augments, Doesn't Replace:** Human oversight (review agent) ensures safety and quality

10.2 Next Steps

Continue learning by:

- Exploring LangChain's tool/function calling capabilities
- Implementing Retrieval-Augmented Generation (RAG)
- Experimenting with different LLMs and comparing results
- Building production-grade error handling and retry logic
- Contributing to open-source LangChain/LangGraph projects

11 References and Resources

11.1 Official Documentation

- LangChain Documentation: https://python.langchain.com
- LangGraph Documentation: https://langchain-ai.github.io/langgraph/
- LangSmith Platform: https://smith.langchain.com
- Ollama Documentation: https://ollama.com/docs

11.2 Tutorials and Guides

- LangChain Academy: https://academy.langchain.com
- Prompt Engineering Guide: https://www.promptingguide.ai

11.3 Community Resources

- LangChain Discord: https://discord.gg/langchain
- GitHub Repository: https://github.com/langchain-ai/langchain

A Appendix A: Sample Outputs

A.1 Example Run Output

```
============================================================
5G NETWORK QUALITY ANALYSIS REPORT
============================================================

KPI SANITY CHECK
VALID

QoE CLASSIFICATION
Poor

NETWORK ADVICE
Your current network performance is experiencing significant
issues. With a latency of 180ms and packet loss of 4.5%,
you may notice buffering during video calls and slow webpage
loading. Here are some suggestions:

1. Move to a location with better signal strength
2. Check if there's network congestion in your area
3. Try switching to airplane mode for 10 seconds, then back
4. Contact your network operator if issues persist
5. Consider using WiFi if available as an alternative
```

```
FINAL VERDICT
User experience is currently NOT satisfactory.
============================================================
```

B Appendix B: Glossary

Agent Autonomous software component that performs specific tasks

dBm Decibel-milliwatts, measure of signal power

Few-Shot Learning Training AI with a small number of examples

KPI Key Performance Indicator

Latency Time delay in network communication

LCEL LangChain Expression Language for chaining operations

LLM Large Language Model (e.g., GPT, Llama)

Ollama Tool for running LLMs locally

Packet Loss Percentage of data packets that fail to reach destination

QoE Quality of Experience, subjective user perception

RSSI Received Signal Strength Indicator

State Dictionary containing workflow data in LangGraph

Temperature Parameter controlling LLM randomness (0=deterministic, 1=creative)

End of Laboratory Manual